

Phase 2: Project Structure Generation

1. System Analysis

System Type: Web-based Application

Features:

- User authentication
- Product management
- Order tracking
- Admin management
- Reporting

Non-Functional Requirements:

- Security (encryption, strong password policies)
- Performance (not specified)
- Usability
- Reliability
- Maintainability
- Portability

Complexity: Medium

2. Architecture

Style: Layered

Pattern: Multi-tiered

Justification:

Given the medium complexity, non-distributed nature, and common features of a web-based application (user authentication, product management, order tracking, admin management, reporting), a layered architecture is most appropriate. This architecture style promotes modularity and simplifies development, testing, and maintenance. The Multi-tiered design pattern within this style can further help in separating presentation, business logic, and data access layers for improved separation of concerns.

3. Tech Stack

Frontend: React.js (for its component-based structure, performance, and large community support)

Backend: Node.js with Express.js (for JavaScript compatibility across frontend and backend, non-blocking I/O approach, and extensive libraries)

Database: PostgreSQL (for its robustness, ACID compliance, and scalability)

Tools:

- JWT for authentication
- Sequelize ORM

Justification:

React.js is a popular choice for building the frontend due to its performance and component-based structure which simplifies complex applications like this one. For the backend, Node.js with Express.js provides an efficient non-blocking I/O approach and JavaScript compatibility, making it easier to manage APIs and handle server logic. PostgreSQL is a reliable database choice for

handling the required transactions and providing scalability as the application grows. JWT (JSON Web Tokens) can be used for secure user authentication, while Sequelize ORM will help in managing interactions with the database more effectively.

4. Project Structure

```

...
/my-mern-app
  /backend
    /config
      database.js
      jwt.js
    /controllers
      authController.js
      productController.js
      orderController.js
      adminController.js
      reportController.js
    /models
      User.js
      Product.js
      Order.js
      Admin.js
      Report.js
    /services
      authenticationService.js
    /middlewares
      authMiddleware.js
  app.js
  /frontend
    public
      index.html
    src
      components
        Auth
          LoginPage.js
          RegisterPage.js
        Products
          ProductList.js
          ProductDetail.js
        Orders
          OrderList.js
          OrderDetail.js
        Admin
          Dashboard.js
        Reports
          ReportView.js
      pages
        HomePage.js
        App.js
        index.js
  .gitignore
  package-lock.json
  package.json
  ...
```